

# ACE PLAY

This project aims to create a Python-based multiplayer card game using client-server architecture. It manages card distribution, rule enforcement, and winner determination with a unique scoring system. A Tkinter-based GUI displays player hands and scores, while socket programming ensures real-time gameplay. A MySQL database stores player info and game results. The system is scalable for future updates, including AI opponents and new game modes

## About the game

We accept at most 5 players to start the game.

There are 52 cards in the deck. A to K of **Spade, Heart, Diamond and Club**.

## Rules of the game

- Every player got 2 cards and has the right to draw one extra card.
  - The value of the card ranks A=1, 2=2, 3=3, 4=4, 5=5, 6=6, 7=7, 8=8, 9=9, 10=10, J=10, Q=10 and K=10.
  - The value of the card depends on the combination of the cards together. If the score is higher than 10, you must ignore the 2nd digit. For example, 15= 5, 14=4, 10=0, 20=0, 33=3.
  - To win the game, you must get the higher score. In case the score is equal in value, then the hand who gets less card is the winner.
  - If the score and number of cards are equal, the value of the card decides the winner. The orders are A, K, Q, J, 10,...2. If the value is the same, look at the suits of the highest value of the card. The orders is Spade, Heart, Diamond and Club.
- => For example: Deck (5,5,7) VS deck (3,4), deck (3,4) is the winner.
- => For example: Deck (Q,6) VS deck (2,4), deck (Q,6) is the winner.
- => For example: Deck (Heart of 7, Club of 2) VS deck (Diamond of 7, Spade of 2), the first deck is the winner.

## How to Run this project

For Server side:

### 1. Requirements:

- Python 3.x installed.
- mysql.connector, PIL, and tkinter modules.
- A MySQL database setup with card image paths.

### 2. MySQL Database Setup:

- Create a database named “ace\_play” with a table card\_images that stores card ranks, suits, and their corresponding image paths.

- Example MySQL table creation:

```
CREATE TABLE card_images (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    card_rank VARCHAR(10),  
    suit VARCHAR(10),  
    image_path VARCHAR(255)  
);
```

- Insert card data into the table (e.g., Ace, Hearts, ace\_of\_hearts.png).

-Create a database named "ace\_play" with a table users that stores card usernames, email, and user passwords.

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(30),  
    email VARCHAR(50),  
    password VARCHAR(50)  
);
```

### 3. Server Configuration:

- The server will run on a local machine (IP: 127.0.0.1, Port: 8000) and handle up to 5 players.
- It includes a game logic (Game class), database connection for fetching card image paths, and broadcast functionality to communicate with clients.

### 4. Running the Server:

- Start the server by running the server.py file. The server will ask for the number of players.
- Once the number of players is reached, the game will start.

## ### Instruction Manual for the Python Card Game

### For Client Side:

#### 1. Requirements:

- Python 3.x installed.
- mysql.connector, PIL, and tkinter modules.

#### 2. Connecting to the Server:

- Players will use the client application (main\_window.py) to connect to the server.
- Each player must enter their email, which fetches their username from the MySQL database.
- Once all players are connected, the game begins.

## **How to Play:**

### **Objective:**

- The goal of the game is to draw cards and create the strongest hand based on the game's rules. The winner is determined by comparing all players' hands.

### **Game Flow:**

#### 1. Starting the Game:

- Players are prompted to connect to the server. Once all players join, the server will start the game automatically.

#### 2. Drawing Cards:

- Each player is initially dealt two cards.
- The cards are displayed on the player's screen with images fetched from the MySQL database.

#### 3. Decision to Draw More Cards:

- After the initial deal, players are asked whether they want to draw one more card (by clicking "OK" for yes, or "Cancel" for no).
- If a player chooses to draw another card, it will be displayed in their frame.

#### 4. Broadcasting Card Details:

- Each player's card details are broadcast to all other players.
- The details are displayed in a scrollable window, showing each player's username and card images.

## 5. Announcing the Winner:

- Once all players have finished drawing cards, the server calculates the winner based on the game logic (strongest hand).
- The winner is broadcast to all players, and the game ends.

## **Controls:**

### 1. Client GUI:

- The client interface uses Tkinter for displaying the game. It includes:
  - Player Name: Displayed at the top of each player's section.
  - Card Images: Shown next to each player's name.
  - Prompt for Another Card: A pop-up appears after the initial deal, asking if the player wants to draw another card.
  - Winner Announcement: Displayed at the end of the game.

### 2. Buttons:

- OK: Accepts the prompt to draw another card.
- Cancel: Declines the prompt and ends the player's turn.

---

## **Server and Client Code Summary:**

### 1. Server (`server.py`):

- Manages the game logic and handles client connections.
- Deals cards, broadcasts messages, and calculates the winner.
- Uses a barrier to synchronize clients before broadcasting results.

### 2. Client (`client.py`):

- Connects to the server and interacts with the player via a GUI.
- Receives and displays card images fetched from the server.
- Handles responses for drawing additional cards.

## **Troubleshooting:**

### Connection Issues:

- Ensure the server is running before starting the client.
- Check that the client is using the correct IP address (127.0.0.1 for local testing).

### Database Errors:

- Ensure MySQL is running, and the database contains correct card image paths.

### Message Merge Issue:

- This has been resolved by using a message delimiter (`\nEND\n`) to ensure messages are processed correctly.

### **Potential Additional Features:**

- The game can be extended with different card games or additional functionality like scores or game statistics.
- Multiple game modes can be implemented in the future